

API Security

An API is reachable by anyone who can find it. Security is the set of answers to four questions: can the traffic be read on the way, who is calling, what are they allowed to do, and what may the page in a browser be trusted with.

This follows on from the API document, which covers the styles themselves.

A good order to learn it in:

SSL and TLS → Authentication methods → JWT → OAuth → CORS → CSP

1. SSL and TLS

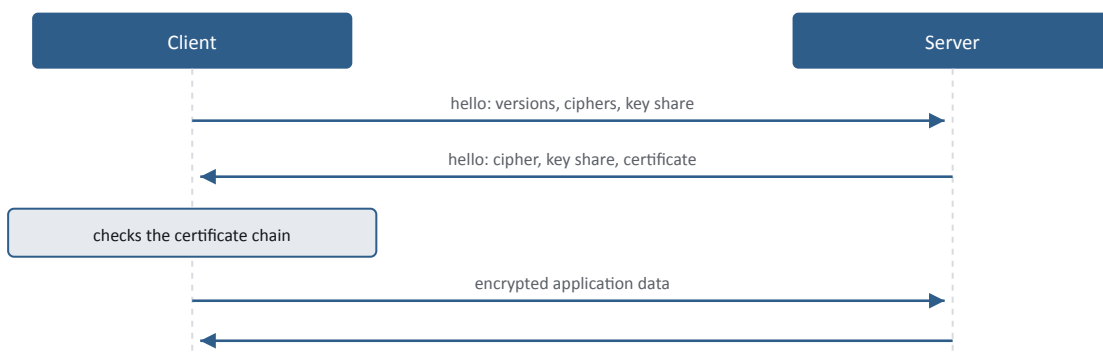
TLS (Transport Layer Security) encrypts the connection between a client and a server, so nobody in between can read or change what is sent.

SSL (Secure Sockets Layer) is the older name for the same job. Every SSL version is broken and switched off; what runs today is TLS 1.2 or 1.3. The name SSL stayed in use anyway, which is why certificates are still called SSL certificates.

It provides three things:

Property	What it means
Encryption	Nobody in between can read the traffic
Integrity	Nobody in between can change it without being noticed
Authentication	The client knows the server is who it claims to be

HTTPS is simply HTTP carried over TLS.



Both sides derive the same session key from the two key shares. The key itself is never sent.

TLS 1.3 completes this in one round trip. TLS 1.2 needed two, which is most of why 1.3 connects faster.

The certificate is what proves identity. It carries the domain it is valid for, the server's public key, the dates it is valid between, and the authority that issued it. The client checks it against a chain:

```
Root CA → Intermediate CA → The server's certificate  
(trusted by the operating system or browser)
```

Most failures are one of four things:

Error	Cause
Certificate expired	It is past its validity date
Hostname mismatch	The certificate is for a different domain
Self-signed certificate	No authority the client already trusts
Incomplete chain	The server did not send the intermediate certificate

The last one is the awkward case, because a browser often has the intermediate cached and succeeds while a server-side client fails on the same endpoint.

mTLS makes it mutual. The client presents a certificate too, so both sides are verified. This is common between internal services, where there is no user to sign in and both ends are ours.

HSTS removes the plain HTTP attempt.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

After the browser sees that header once, it upgrades to HTTPS by itself, so a request to `http://` never leaves the machine to be intercepted.

Two limits worth stating. TLS protects data while it moves, not where it is stored, and not from the server itself. And terminating TLS at a load balancer means the traffic behind it is plain text unless it is encrypted again.

```
TLS → nobody between the client and the server can read or change the traffic
```

2. Authentication methods

Two words that are easy to mix up:

```
Authentication → who is calling  
Authorization → what they are allowed to do
```

Every request has to carry proof of who is calling. These are the ways to carry it.

Method	What the client sends	Where it fits
Basic	Username and password, every request	Internal tools and scripts
API key	A fixed secret string	One service calling another
Bearer token	A token issued after signing in	Most modern APIs
Cookie and session	A session id the browser stores	Server-rendered web apps
mTLS	A client certificate	Service to service inside a network

Basic authentication puts the username and password in a header on every request.

```
GET /employees/1 HTTP/1.1
Authorization: Basic cmF2aTpzM2NyZXQ=
```

That value is Base64 of `ravi:s3cret`. Base64 is encoding, not encryption, so anyone who sees the header has the password. It is only acceptable over HTTPS, and even then the password travels on every single call.

When no credentials arrive, the server asks for them:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="employees"
```

An API key is a fixed secret that identifies an application rather than a person.

```
GET /employees/1 HTTP/1.1
X-API-Key: ak_live_9f2b7c41
```

It does not expire on its own, so it has to be scoped to what it needs, rotated on a schedule, and kept out of URLs. A key in a query string ends up in server logs, browser history and referrer headers.

A bearer token is issued once the user has signed in, and sent on each request afterwards.

```
GET /employees/1 HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

"Bearer" is literal: whoever holds the token can use it. The token is usually a JWT, which the next section covers.

Cookie and session authentication keeps the state on the server. Signing in creates a session, and the response plants a cookie holding only its id.

```
HTTP/1.1 200 OK
Set-Cookie: SESSIONID=8f3b2a; HttpOnly; Secure; SameSite=Lax; Path=/
```

The browser then attaches it to every request on its own:

```
GET /employees/1 HTTP/1.1
Cookie: SESSIONID=8f3b2a
```

The three flags are the security:

Flag	What it does
HttpOnly	JavaScript cannot read the cookie, so an XSS bug cannot steal it
Secure	The cookie is sent over HTTPS only
SameSite=Lax	The cookie is not sent on most cross-site requests

Token against cookie is the comparison interviews ask for.

```
Cookie: sign in → server stores a session → Set-Cookie → browser returns it
Token:  sign in → server signs a token    → response body → client returns it
```

	Cookie and session	Bearer token
State	Held on the server	Held in the token
Sent by	The browser, automatically	The client, deliberately
Ending a session	Delete it on the server	Hard, the token stays valid
Cross-site request forgery	Exposed, needs a token or SameSite	Not exposed, the header is not sent automatically
Script stealing it	HttpOnly protects it	Readable if stored in localStorage
Mobile and other clients	Awkward	Natural
Scaling out	Needs a shared session store	Any server can verify it

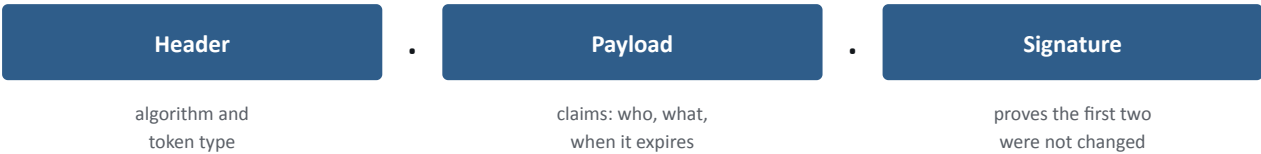
Neither is better in every case. The automatic sending that exposes cookies to request forgery is the same thing that keeps the token away from JavaScript. Many applications take both by putting a JWT inside an HttpOnly cookie.

OAuth is not a fifth item on this list. It is how a client gets a bearer token when the data belongs to a user on another service. Once it holds one, it sends it the same way as any other bearer token.

3. JWT

JWT (JSON Web Token) is a token used to securely transmit information and authenticate users.

It is three parts joined by dots.



Each part is Base64url encoded. The first two are readable by anyone who has the token.

A token on the wire:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODFiIiwiaWF0IjoxNTE2MzkwMjQ0LCJpc29udCI6Im5hbWUiOiJJSYXZpIiwicm9sZSI6ImFkbWwIn0.TJVA950rM7E2cBab30RMHrHDcEfxjoYZg
```

The first two parts decode to plain JSON:

```
{ "alg": "HS256", "typ": "JWT" }

{
  "sub": "12345",
  "name": "Ravi",
  "role": "admin",
  "iat": 1769790143,
  "exp": 1769876543
}
```

The claims in the payload have agreed names:

Claim	Meaning
sub	Subject. Who the token is about
iss	Issuer. Who created it
aud	Audience. Which service it is for
exp	Expiry time
iat	Issued at
nbf	Not valid before
jti	Token id, used to revoke a single token

The client sends it on every request:

```
GET /employees/1 HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Creating one in Java:

```
String token = Jwts.builder()
    .subject("12345")
    .claim("role", "admin")
    .expiration(Date.from(Instant.now().plus(Duration.ofHours(1))))
    .signWith(key)
    .compact();
```

A JWT is signed, not encrypted. The signature proves nobody changed the contents. It does not hide them. Anyone holding the token can read the payload, so a password, a card number or anything private does not belong in it.

The server keeps no session. It verifies the signature with a key it already has, which is why a JWT scales across many servers with no shared store. That is the benefit, and the cost is the next point.

A JWT cannot be taken back. Once issued it stays valid until `exp`. Logging out on the client does not stop the token working. The usual answers are a short expiry with a refresh token, or a deny list of `jti` values, which brings back the lookup the token was meant to avoid.

Check the algorithm. A server that trusts the `alg` field in the header can be handed a token saying `"alg": "none"`. Verify against the algorithm the server expects, not the one the token claims.

JWT → signed facts about a user, readable by anyone, valid until it expires

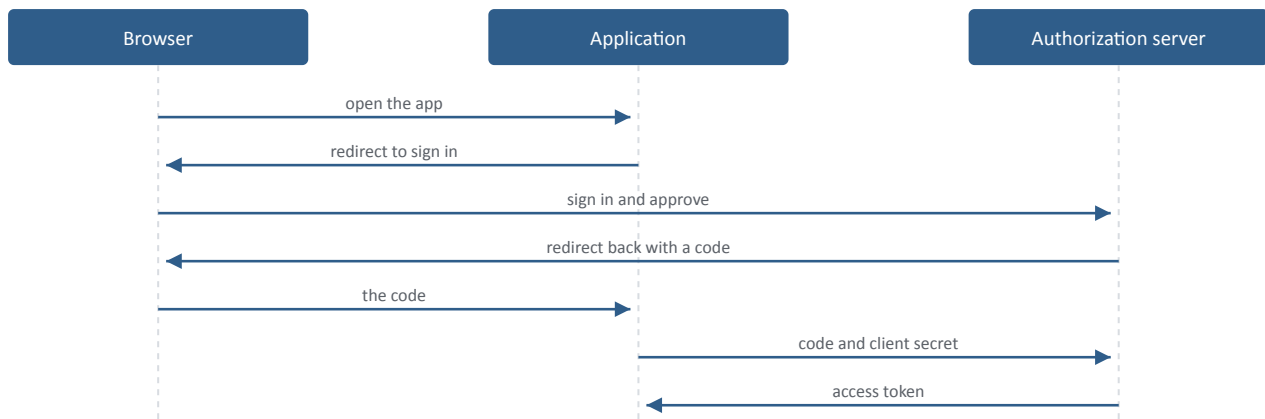
4. OAuth

OAuth (Open Authorization) allows an application to access a user's resources without sharing their password.

Four roles take part:

Role	Who it is
Resource owner	The user who owns the data
Client	The application asking for access
Authorization server	Signs users in and issues tokens
Resource server	Holds the data and accepts tokens

The common flow is the authorization code flow:



The code travels through the browser. The secret and the token never do.

That split is the reason for the flow. The short-lived code can pass through the browser safely, because it is useless without the client secret, which stays on the server.

Grant types are the different ways a client gets a token. Which one to use depends on what the client is.

Grant type	Used by	Notes
Authorization Code	Server-side web apps	The default for anything with a user
Authorization Code with PKCE	Mobile apps, single page apps	Adds a proof step, since these cannot keep a secret
Client Credentials	One service calling another	No user involved
Refresh Token	Any client with a refresh token	Gets a new access token without asking the user again
Device Code	TVs, consoles, devices with no keyboard	The user approves on a phone instead
Implicit	Nothing new	Deprecated. It returned the token in the URL
Resource Owner Password	Nothing new	Deprecated. The client saw the password

PKCE, Proof Key for Code Exchange, is the detail interviews ask about. A public client cannot hide a secret, so it sends a hashed random value up front and the original value when exchanging the code. An attacker who steals the code cannot use it.

OAuth is authorization, not authentication. It answers "may this app read the user's calendar", not "who is this user". OpenID Connect sits on top of OAuth and adds an ID token, which is a JWT describing the user. "Sign in with Google" is OpenID Connect, not plain OAuth.

OAuth → an app gets limited access without the user's password
OIDC → OAuth plus an ID token that says who the user is

5. CORS

CORS (Cross-Origin Resource Sharing) allows a web browser to access resources from a different origin.

An origin is the scheme, the host and the port together. All three must match:

```
https://app.example.com      and https://api.example.com    → different
https://app.example.com      and http://app.example.com     → different
https://app.example.com      and https://app.example.com:8443 → different
```

The rule exists because a browser attaches the user's cookies automatically. A page on a hostile site could otherwise call a bank's API with the user's session and read the answer.

For a plain `GET`, the browser adds an `Origin` header and checks the reply:

```
GET /employees/1 HTTP/1.1
Origin: https://app.example.com
```

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://app.example.com
```

Without that header the response is blocked, and the page sees an error even though the server ran the request.

Anything beyond a simple request gets a preflight. A custom header, a `PUT` or `DELETE`, or a JSON content type makes the browser ask permission first with an `OPTIONS` request:

```
OPTIONS /employees/1 HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: Authorization, Content-Type
```

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Max-Age: 86400
```

Only after that does the real `PUT` go out. `Access-Control-Max-Age` tells the browser how long it may skip asking again.

Header	What it does
<code>Access-Control-Allow-Origin</code>	Which origin may read the response
<code>Access-Control-Allow-Methods</code>	Which methods are allowed
<code>Access-Control-Allow-Headers</code>	Which request headers are allowed
<code>Access-Control-Allow-Credentials</code>	Whether cookies may be sent
<code>Access-Control-Max-Age</code>	How long a preflight result may be reused

Three points to remember:

The browser enforces CORS, not the server. `curl` and Postman ignore it completely, which is why an endpoint works in a terminal and fails in a page.

It is not a defense against an attacker. It protects the user's browser session. Anything that is not a browser can call the endpoint regardless, so the API still needs real authentication.

`Access-Control-Allow-Origin: *` **cannot be combined with credentials.** With `Allow-Credentials: true` the origin has to be named exactly.

CORS → the browser asking the server whether this page may read the answer

6. CSP

CSP (Content Security Policy) is a response header that tells the browser which sources a page may load scripts, styles and other resources from.

It exists to limit cross-site scripting. Even when an attacker manages to inject a `<script>` tag, the browser refuses to run it unless the policy allows that source.

```
Content-Security-Policy: default-src 'self';
  script-src 'self' https://cdn.example.com;
  connect-src 'self' https://api.example.com;
  img-src 'self' data;;
  frame-ancestors 'none'
```

The directives name a kind of resource:

Directive	What it controls
<code>default-src</code>	The fallback for anything not named separately
<code>script-src</code>	Where scripts may be loaded from
<code>style-src</code>	Where stylesheets may be loaded from

Directive	What it controls
<code>img-src</code>	Where images may be loaded from
<code>connect-src</code>	Where <code>fetch</code> , <code>XMLHttpRequest</code> and WebSockets may go
<code>form-action</code>	Where a form may submit
<code>frame-ancestors</code>	Who may place this page inside a frame

The values name a source:

Value	Meaning
<code>'self'</code>	The page's own origin
<code>'none'</code>	Nothing at all
<code>https://cdn.example.com</code>	That exact origin
<code>data:</code>	Data URLs, used for small inline images
<code>'unsafe-inline'</code>	Inline scripts and styles. It removes most of the protection
<code>'nonce-a1b2c3'</code>	Only tags carrying this one-time value

Report-only mode is how a policy gets introduced. Nothing is blocked, and every violation is reported, so a real policy can be written from what the site actually loads.

```
Content-Security-Policy-Report-Only: default-src 'self'; report-uri /csp-report
```

`frame-ancestors 'none'` is also the modern replacement for `X-Frame-Options`, which is what stops another site framing the page and tricking a user into clicking something.

CSP and CORS are both browser rules, and they answer opposite questions.

	CORS	CSP
The question	May this page read a response from another origin	May this page load a resource from this source
Set by	The server being called	The server that served the page
Protects	An API's data from other pages	A page's users from injected content
Header	<code>Access-Control-Allow-Origin</code>	<code>Content-Security-Policy</code>

CORS → what this page may read
CSP → what this page may load and run

The pieces together

Piece	What it answers
TLS	Can anyone between us read or change this
Authentication method	How a request proves who is calling
JWT	What do we know about this user, without a lookup
OAuth	May this app act for this user, without their password
OpenID Connect	Who is this user
CORS	May this page read this response
CSP	May this page load and run this resource

They stack rather than compete. TLS carries everything else, so without it the rest is written on a postcard. Authentication decides whose request this is. OAuth is one way to obtain the token that authentication then checks. CORS and CSP apply only in a browser, and they protect different things.

For example, imagine a food delivery application. Every connection is HTTPS, and HSTS means a plain `http://` attempt never leaves the phone. Signing in uses OpenID Connect with the authorization code flow and PKCE, because a mobile app cannot keep a secret. Every call after that carries a JWT in an `Authorization` header, and each service verifies it without a database lookup. The internal dashboard is served from a different origin from the API, so the API returns `Access-Control-Allow-Origin` for that one origin and no other, and the dashboard itself sends a CSP that allows scripts only from its own origin and one CDN.

The most useful thing at this point is not to memorize the headers. Paste a real token into a JWT decoder and read the claims, which is the quickest way to see that a JWT hides nothing. Then open any site in a browser and look at its response headers for `Strict-Transport-Security` and `Content-Security-Policy`.